NGO Donation Management System

by

KIRUTHIKA S

PROBLEM DEFINITION

Business Context

Q1. What problem are we solving?

The NGO does not have a single system to manage donors, donations, and campaigns. Most records are kept manually or in different files, which causes confusion, errors, and difficulty in tracking funds.
This system will help the NGO store all donation information in one place and easily track donations, donors, and campaign details.

Q2. Who are the stakeholders?

The main users of the system are:

NGO Management – to plan and manage funds properly
Donors – to check their donation history and receipts
Campaign Managers – to track campaign progress
Finance Team – to manage accounts and reports
IT/Admin Team – to maintain system security and data accuracy

Q3. What decisions will this system support?

The system will help in:

Finding which campaigns receive more or fewer donations
Understanding donor donation patterns
Planning future fundraising activities
Ensuring transparency in fund usage

Cross Validation

Data should be well-organized and properly linked
No missing or incorrect records
Secure and reliable database design

REQUIREMENTS ENGINEERING

Functional Requirements

1. Register donors
2. Register NGOs / Project
3. Record donations
4. Assign donations to projects
5. Generate donation reports

Business Rules

1. A donor can make multiple donations
2. Each donation must be linked to one NGO/project
3. Donation amount must be a valid positive value

Cross Validation

Every business rule must be enforceable by:

Database schema
Constraints
Application logic

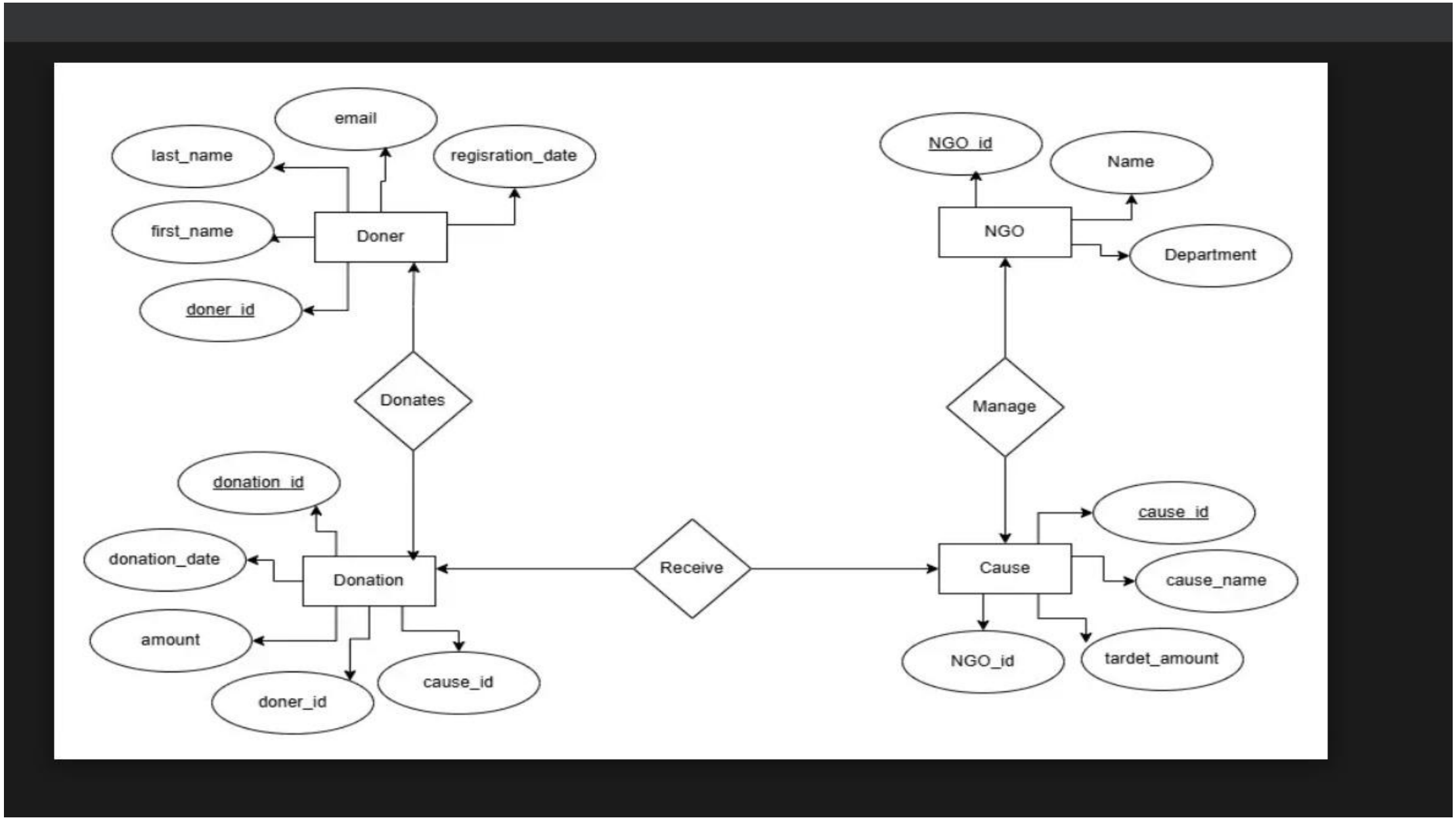
CONCEPTUAL DATA MODEL (ERD)

Identify Entities	Identify Relationships	Attributes & Keys
Donor Cause NGO Donation (Junction Table)	Donor — Cause → **M:N** NGO —Cause→ **1:N**	Donor_id (PK) Cause_id(PK) NGO_id (PK) Donation_id (PK) Ensure atomic attributes No derived data

Cross Validation

1. No multi-valued attributes
2. Each entity has a PK
3. Attributes atomic
4. Relationships clearly defined
5. No attributes stored incorrectly
6. Use Donation table to manage relationships

CONCEPTUAL DATA MODEL (ERD)



LOGICAL DATA MODEL

RESOLVE M:N RELATIONSHIPS (IF ANY)

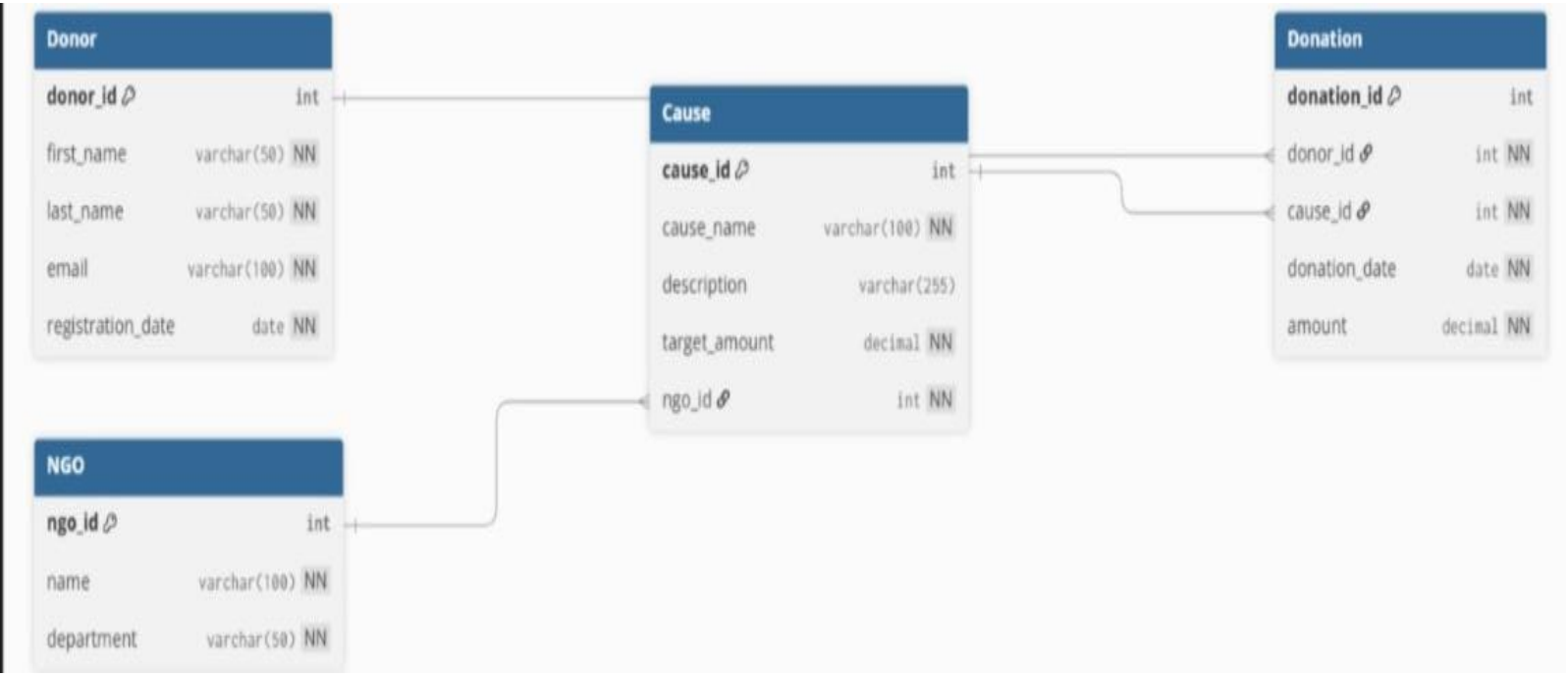
1. Donation junction table

Constraints

1. A donor cannot donate twice for the same cause on the same date **UNIQUE(donor_id, cause_id, donation_date)**
2. A donation must be linked to one donor
3. A donation must be linked to one

DEFINE CARDINALITY & PARTICIPATION

Relationship	Cardinality	Degree	Participation
Donor→ Donation	1:N	Binary	Partial
Donation→Donar	N:1	Binary	Total
Cause→ Donation	N:1	Binary	Total



Normalize	Status
1NF: All attributes atomic	Verified
2NF: No partial dependency (no composite PK misuse)	Verified
3NF: No transitive dependency (NGO info NOT in Cause table beyond FK)	Verified

Cross Validation

No multi-valued attributes
Each entity has a PK
Attributes atomic
M:N relationship identified explicitly
No attributes stored on relationships incorrectly
Resolve M:N using Junction Table

PHYSICAL MODEL (SQL)

```sql
CREATE TABLE Donor (
    donor_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100) UNIQUE,
    registration_date DATE
);
CREATE TABLE NGO (
    ngo_id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50)
);
CREATE TABLE Cause (
    cause_id INT PRIMARY KEY,
    cause_name VARCHAR(100),
    target_amount INT,
    ngo_id INT,
    FOREIGN KEY (ngo_id) REFERENCES NGO(ngo_id)
);
CREATE TABLE Donation (
    donation_id INT PRIMARY KEY,
    donor_id INT,
    cause_id INT,
    donation_date DATE,
    amount INT,
    FOREIGN KEY (donor_id) REFERENCES Donor(donor_id),
    FOREIGN KEY (cause_id) REFERENCES Cause(cause_id),
    UNIQUE (donor_id, cause_id)
);
CREATE INDEX idx_donation_donor ON Donation(donor_id);
CREATE INDEX idx_donation_cause ON Donation(cause_id);
```

Cross Validation

Referential integrity enforced
Business rule enforced via UNIQUE constraint
Indexes align with query patterns
FK constraints prevent orphan records
No redundant data

DATA INGESTION & TESTING

Test with Sample Data

```sql
INSERT INTO Donor VALUES(1, 'Alice', 'Wong', 'alice@donors.org', '2024-01-01');
INSERT INTO NGO VALUES(1, 'Helping Hands NGO', 'Fundraising');
INSERT INTO Cause VALUES(101, 'Clean Water Drive', 100000, 1);
INSERT INTO Donation VALUES(1001, 1, 101, '2024-01-15', 500);
```

Data Quality Checks – Refer SQL Codes in Appendix

1. **Referential Integrity (Must return ZERO rows)**
   Donation → Donor (FK check)
   Donation → Campaign (FK check)
2. **Domain Enforcement**
   Donation amount > 0
   Campaign start_date < end_date
   Mandatory fields are NOT NULL
3. **Constraint Violation Tests**
   Duplicate donation detection
   Primary key uniqueness
   Foreign key participation
4. **Logical Consistency**
   Checks Campaigns with no donations
   Donors with no donations
   Donations outside campaign date
5. **Negative Test Cases (Should FAIL)**
   Duplicate donation entry
   Negative donation amount

QUERY & ANALYTICS LAYER

Create a view to expose data to user

BUSINESS QUERY 1:

```
USE college_db;
CREATE OR REPLACE VIEW vw_campaign_donation_analytics AS
SELECT
    campaign_id,
    campaign_name,
    COUNT(donation_id) AS total_donations,
    ROUND(AVG(donation_amount), 2) AS avg_donation
FROM vdonationanalytics
GROUP BY campaign_id, campaign_name;
SELECT *
FROM vw_campaign_donation_analytics
ORDER BY total_donations DESC
LIMIT 3;
SELECT *
FROM vw_campaign_donation_analytics
ORDER BY avg_donation DESC;
```

BUSINESS QUERY 2:

```
CREATE VIEW v_campaign_avg_donation AS
SELECT
    campaign_id,
    campaign_name,
    ROUND(AVG(donationamount), 2) AS avg_donation
FROM vdonationanalytics
GROUP BY
    campaign_id,
    campaign_name;
SELECT *
FROM v_campaign_avg_donation
ORDER BY avg_donation DESC;
```

APPENDIX – SQL Code

```
Table Donor {
  donor_id int [pk]
  first_name varchar(50) [not null]S
  last_name varchar(50) [not null]
  email varchar(100) [not null, unique]
  registration_date date [not null]
}

Table NGO {
  ngo_id int [pk]
  name varchar(100) [not null]
  department varchar(50) [not null]
}

Table Cause {
  cause_id int [pk]
  cause_name varchar(100) [not null]
  description varchar(255)
  target_amount decimal [not null]
  ngo_id int [not null]
}

Table Donation {
  donation_id int [pk]
  donor_id int [not null]
  cause_id int [not null]
  donation_date date [not null]
  amount decimal [not null]
}
Ref: Donor.donor_id < Donation.donor_id
Ref: Cause.cause_id < Donation.cause_id
Ref: NGO.ngo_id < Cause.ngo_id
```